



TP02 : Prise en main de Hadoop

Description

Le Namenode

Le Namenode maintient l'arborescence du système de fichiers et les métadonnées de l'ensemble des fichiers et répertoires d'un système Hadoop. Toutes ces métadonnées, hormis la position des blocs dans les Datanodes, sont stockées physiquement sur le disque système dans deux types de fichiers spécifiques **edits_xxx** et **fsimage_xxx**.

Ce PC > Windows (C:) > hadoop-3.2.3 > data > namenode > current				
Nom	Modifié le	Type	Taille	
edits_00000000000000000001-0000000000...	20/04/2022 17:07	Fichier	1 024 Ko	
edits_00000000000000000002-0000000000...	20/04/2022 17:21	Fichier	1 024 Ko	
edits_00000000000000000003-0000000000...	20/04/2022 22:01	Fichier	1 024 Ko	
edits_inprogress_00000000000000000016	20/04/2022 22:31	Fichier	1 024 Ko	
fsimage_00000000000000000002	20/04/2022 18:28	Fichier	1 Ko	
fsimage_00000000000000000002.md5	20/04/2022 18:28	Fichier MD5	1 Ko	
fsimage_000000000000000000015	20/04/2022 22:19	Fichier	1 Ko	
fsimage_000000000000000000015.md5	20/04/2022 22:19	Fichier MD5	1 Ko	
seen_txid	20/04/2022 22:19	Fichier	1 Ko	
VERSION	20/04/2022 22:19	Fichier	1 Ko	

La connaissance de la position des blocs dans les Datanodes est reconstruite à chaque démarrage du Namenode dans un mode appelé **safe mode**. Pendant le safe mode, l'écriture sur HDFS est impossible, le Namenode charge les fichiers edits_xxx et fsimage_xxx et attend le retour des Datanodes sur la position des blocs. Une fois toutes les opérations réalisées, le safe mode est relâché et l'accès en écriture est de nouveau autorisé. Soyez patient sur la durée du safe mode. Celui-ci peut être très long si vous avez beaucoup de fichiers à traiter.

Le Secondary Namenode

Son fonctionnement est relativement simple puisque le Namenode secondaire vérifie périodiquement l'état du Namenode principal et copie les métadonnées via les fichiers edits_xxx et fsimage_xxx.



Les Datanodes

Les Datanodes sont sous les ordres du Namenode et sont surnommés les Workers. Ils sont donc sollicités par les Namenodes lors des opérations de lecture et d'écriture. En lecture, les Datanodes vont transmettre au client les blocs correspondant au fichier à transmettre. En écriture, les Datanodes vont retourner l'emplacement des blocs fraîchement créés. Les Datanodes sont également sollicités lors de l'initialisation du Namenode et aussi de manière périodique, afin de retourner la liste des blocs stockés.

MapReduce

Le modèle de programmation MapReduce fournit un cadre à un développeur afin d'écrire une fonction de Map et de Reduce. Le développeur n'a pas à se soucier du travail de parallélisation et de distribution du travail, de récupération des données sur HDFS, de développements spécifiques à la couche réseaux pour la communication entre les nœuds, ou d'adapter son développement en fonction de l'évolution de la montée en charge (scalabilité horizontale, par exemple). Ainsi, le modèle de programmation permet au développeur de ne s'intéresser qu'à la partie algorithmique. Il transmet alors son programme MapReduce développé dans un langage de programmation au framework Hadoop pour l'exécution.

Le terme de « job » MapReduce est couramment utilisé dans la littérature. Celui-ci concerne une unité de travail que le client souhaite réaliser. Cette unité est constituée de données d'entrée (contenues dans HDFS), d'un programme MapReduce (implémentation des fonctions map et reduce) et de paramètres d'exécution. Hadoop exécute ce job en le subdivisant en deux tâches : les tâches de **map** et les tâches de **reduce**.

Avant l'exécution des tâches reduce, deux opérations intermédiaires doivent être exécutées pour préparer la valeur de son paramètre d'entrée. La première opération appelée **shuffle** permet de grouper les valeurs dont la clé est commune. La seconde opération appelée **sort** permet de trier par clé. À la différence des fonctions map et reduce, shuffle et sort sont des fonctions fournies par le framework Hadoop. Il n'y a donc pas à les implémenter.

Exercice 1 : Manipulation du DFS

Exécuter les commandes suivantes pour vous familiariser avec le système de fichier distribué de Hadoop.

Démarrage du dfs

```
start-dfs.cmd
```




- Télécharger un fichier

```
hdfs dfs -get /input/file.txt C:\hadoop-3.2.3\data
```

```
C:\WINDOWS\system32>hdfs dfs -get /input/file.txt C:\hadoop-3.2.3\data  
C:\WINDOWS\system32>
```

- Copier un fichier

```
hdfs dfs -cp /input/file.txt /test
```

```
C:\WINDOWS\system32>hdfs dfs -cp /input/file.txt /test  
C:\WINDOWS\system32>
```

- Voir le contenu du fichier (non disponible si la quantité de données est importante)

```
hdfs dfs -cat /input/file.txt
```

```
C:\WINDOWS\system32>hdfs dfs -cat /input/file.txt  
a  
d  
s  
r  
t  
s  
a  
  
C:\WINDOWS\system32>
```

- Déplacer un fichier (Le fichier n'a pas été déplacé physiquement, seules les métadonnées ont changé)

```
hdfs dfs -mv /input/file.txt /temp
```

```
C:\WINDOWS\system32>hdfs dfs -mv /input/file.txt /temp  
C:\WINDOWS\system32>
```

- Supprimer un fichier (Le fichier n'est pas réellement supprimé, il est placé dans la corbeille qui se nettoie automatiquement).



```
hdfs dfs -rm /test/file.txt
```

```
C:\WINDOWS\system32>hdfs dfs -rm /test/file.txt  
Deleted /test/file.txt
```

Exercice 2 : Exécution de Job

Créer le code Java

1. Créer un projet
2. Créer un package
3. Créer dans le package la classe du Mapper (**MyMapper**)
4. Créer dans le package la classe du Reducer (**MyReducer**)
5. Créer dans le package la classe contenant la méthode main (**MyRunner**)

Code de la classe MyMapper

```
package wordcount;

import java.io.IOException;
import java.util.StringTokenizer;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.LongWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapred.MapReduceBase;
import org.apache.hadoop.mapred.Mapper;
import org.apache.hadoop.mapred.OutputCollector;
import org.apache.hadoop.mapred.Reporter;

/*
 * Source https://www.javatpoint.com/mapreduce-word-count-example
 * autre référence : https://data-flair.training/forums/topic/what-is-identity-mapper-and-reducer/
 */
/*
 * MapReduceBase est une classe de base pour le Mapper. Elle fournit des implémentations
 * par défaut pour quelques méthodes, la plupart des applications non triviales doivent
 * surcharger certaines de ces méthodes.
 *
 * Mapper est une interface générique. Elle fait correspondre les paires clé-valeur
 * en entrée à un ensemble de paires clé-valeur intermédiaires.
 */
/**
 * @author mariendiaye
 *
 * MapReduceBase est une classe de base pour le Mapper. Elle fournit des implémentations
 * par défaut pour quelques méthodes, la plupart des applications non triviales doivent
 */
```



```
* surcharger certaines de ces méthodes.
* https://hadoop.apache.org/docs/r2.4.1/api/org/apache/hadoop/mapred/MapReduceBase.html
*
*
* Mapper est une interface générique. Elle fait correspondre les paires clé-valeur
* en entrée à un ensemble de paires clé-valeur intermédiaires.
* http://hadoop4beginner.blogspot.com/2015/02/mapper-interface.html
* https://hadoop.apache.org/docs/r2.7.2/api/org/apache/hadoop/mapred/Mapper.html
*/
public class MyMapper extends MapReduceBase implements
Mapper<LongWritable,Text,Text,IntWritable>{
    /*
    * IntWritable est la classe enveloppe dans Hadoop, similaire à la classe Integer
    * dans Java. Elle est optimisée pour permettre la sérialisation dans Hadoop.
    *
    * https://data-flair.training/forums/topic/what-is-intwritable-in-mapreduce-hadoop/
    */
    private final static IntWritable one = new IntWritable(1);

    /*
    * La classe Text stocke du texte en utilisant l'encodage standard UTF8.
    * Elle fournit des méthodes pour sérialiser, désérialiser et comparer
    * les textes au niveau des octets.
    *
    * https://hadoop.apache.org/docs/r2.6.2/api/org/apache/hadoop/io/Text.html
    */
    private Text word = new Text();

    /*
    * @param LongWritable key
    * LongWritable est la classe enveloppe dans Hadoop, similaire à la classe Long
    * dans Java. Elle est optimisée pour permettre la sérialisation dans Hadoop.
    */

    /*
    * @param OutputCollector<Text,IntWritable> output
    * OutputCollector est la généralisation de la fonction fournie par le framework
    * Map-Reduce pour collecter les données produites par le Mapper ou le Reducer,
    * c'est-à-dire les sorties intermédiaires ou le résultat du travail.
    * Il s'agit d'une interface générique similaire à Map en Java.
    *
    * https://hadoop.apache.org/docs/r2.4.1/api/org/apache/hadoop/mapred/OutputCollector.html
    */
}
```



```
* @param Reporter reporter
* Une fonction permettant aux applications Map-Reduce de signaler leur progression
* et de mettre à jour les compteurs, les informations d'état, etc.
*
* Le Mapper et le Reducer peuvent utiliser le Reporter fourni pour signaler leur
* progression ou simplement indiquer qu'ils sont en vie. Dans les scénarios où
* l'application prend beaucoup de temps pour traiter des paires clé/valeur
* individuelles, c'est crucial car le framework pourrait supposer que la tâche est
* arrivée à terme et la tuer.
*
* Les applications peuvent également mettre à jour les compteurs via le Reporter
* fourni.
*
* https://hadoop.apache.org/docs/r2.7.2/api/org/apache/hadoop/mapred/Reporter.html
*/
public void map(LongWritable key, Text value, OutputCollector<Text, IntWritable> output,
    Reporter reporter) throws IOException{
    String line = value.toString();
    /*
    * La classe StringTokenizer permet à une application de découper
    * une chaîne de caractères en sous-chaînes, en se basant sur une série
    * de délimiteurs. L'ensemble des délimiteurs (les caractères qui séparent
    * les éléments) peut être redéfini à tout moment.
    *
    * Les caractères espace, retour chariot, nouvelle ligne et tabulation sont
    * les délimiteurs par défaut séparant les éléments.
    *
    * https://www.enib.fr/~harrouet/Data/oRis/oRisDocHtml/Ref/StringTokenizer.htm
    */
    StringTokenizer tokenizer = new StringTokenizer(line);

    while (tokenizer.hasMoreTokens()){
        word.set(tokenizer.nextToken());
        output.collect(word, one);
    }
}
```

Code de la classe MyReducer

```
package wordcount;

import java.io.IOException;
import java.util.Iterator;
import org.apache.hadoop.io.IntWritable;
```



```
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapred.MapReduceBase;
import org.apache.hadoop.mapred.OutputCollector;
import org.apache.hadoop.mapred.Reducer;
import org.apache.hadoop.mapred.Reporter;

/*
 * Reducer est une interface qui Réduit un ensemble de valeurs intermédiaires qui
 * partagent une clé à un ensemble plus petit de valeurs.
 *
 * https://hadoop.apache.org/docs/r2.8.0/api/org/apache/hadoop/mapred/Reducer.html
 */
public class MyReducer extends MapReduceBase implements
Reducer<Text,IntWritable,Text,IntWritable> {
    public void reduce(Text key, Iterator<IntWritable>
values,OutputCollector<Text,IntWritable> output,
        Reporter reporter) throws IOException {
        int sum=0;
        while (values.hasNext()) {
            sum+=values.next().get();
        }
        output.collect(key,new IntWritable(sum));
    }
}
```

Code de la classe MyRunner

```
package wordcount;

import java.io.IOException;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapred.FileInputFormat;
import org.apache.hadoop.mapred.FileOutputFormat;
import org.apache.hadoop.mapred.JobClient;
import org.apache.hadoop.mapred.JobConf;
import org.apache.hadoop.mapred.TextInputFormat;
import org.apache.hadoop.mapred.TextOutputFormat;

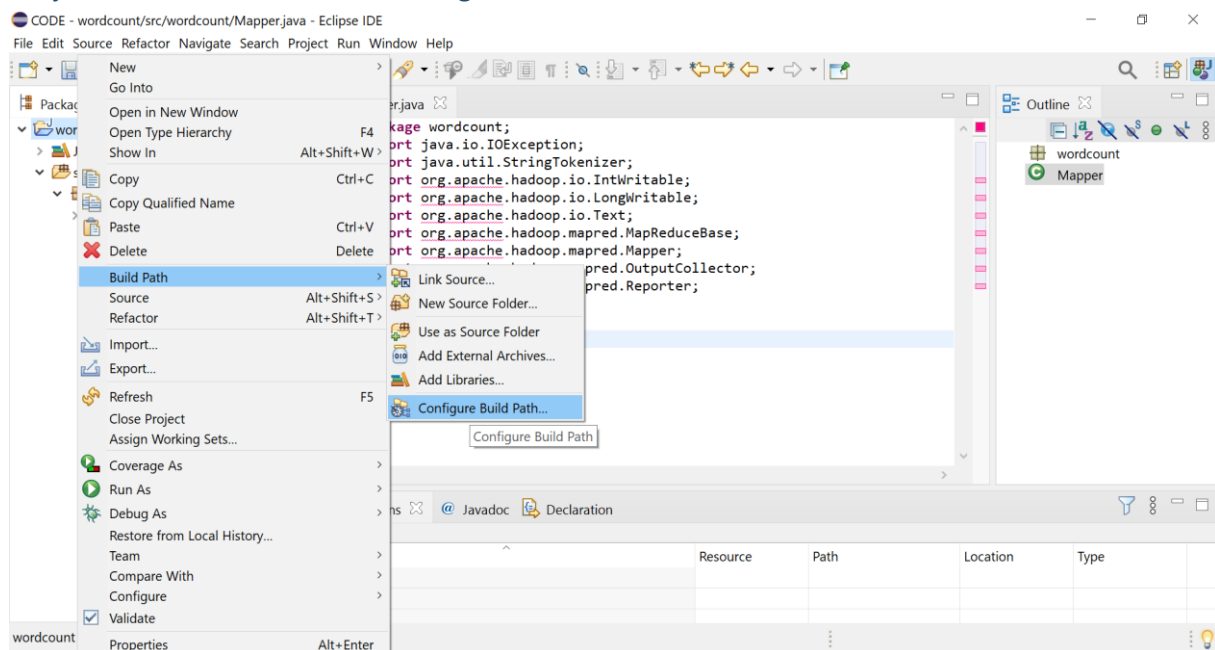
public class MyRunner {
    public static void main(String[] args) throws IOException{
        JobConf conf = new JobConf(MyRunner.class);
        conf.setJobName("WordCount");
        conf.setOutputKeyClass(Text.class);
        conf.setOutputValueClass(IntWritable.class);
        conf.setMapperClass(MyMapper.class);
    }
}
```




```
conf.setCombinerClass(MyReducer.class);
conf.setReducerClass(MyReducer.class);
conf.setInputFormat(TextInputFormat.class);
conf.setOutputFormat(TextOutputFormat.class);
FileInputFormat.setInputPaths(conf,new Path(args[0]));
FileOutputFormat.setOutputPath(conf,new Path(args[1]));
JobClient.runJob(conf);
    }
}
```

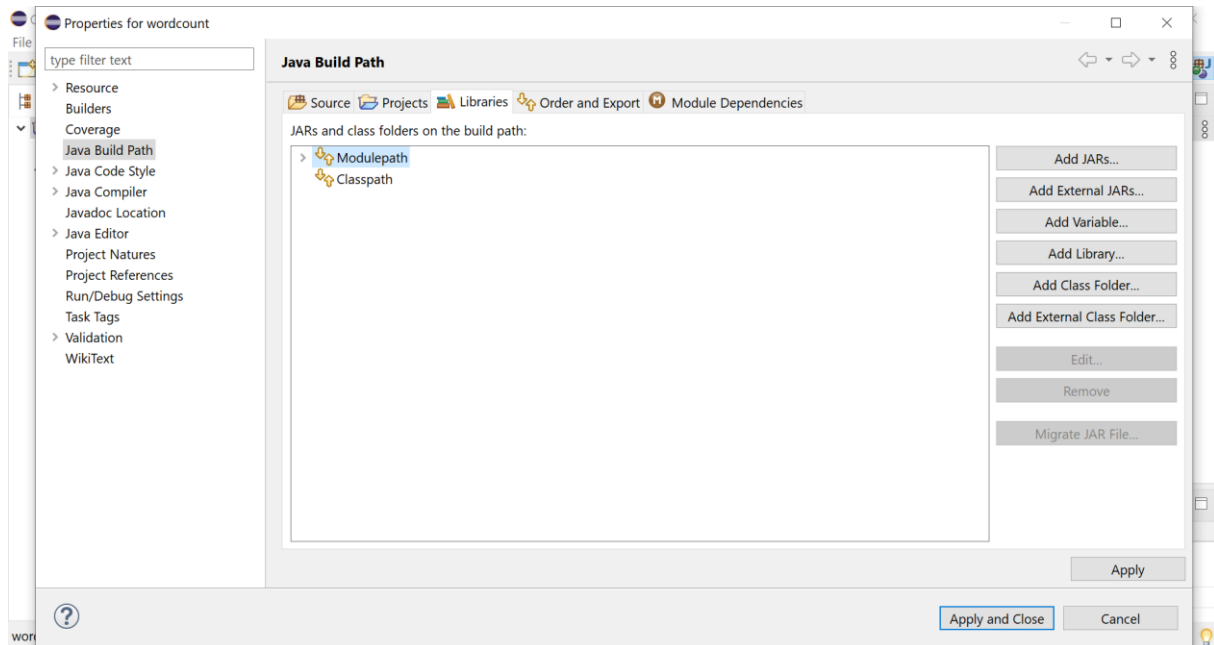
Importer les librairies de Hadoop

Project Name >>Build Path>> configure Build Path.

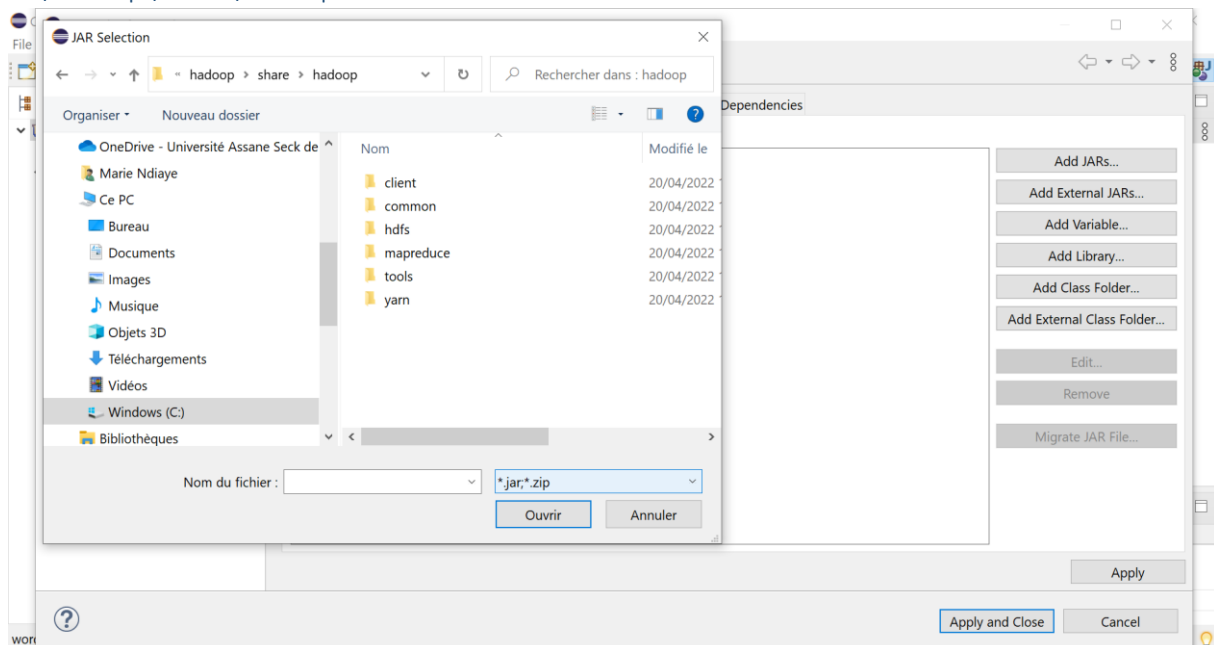




Librairies >> Add the External JARs

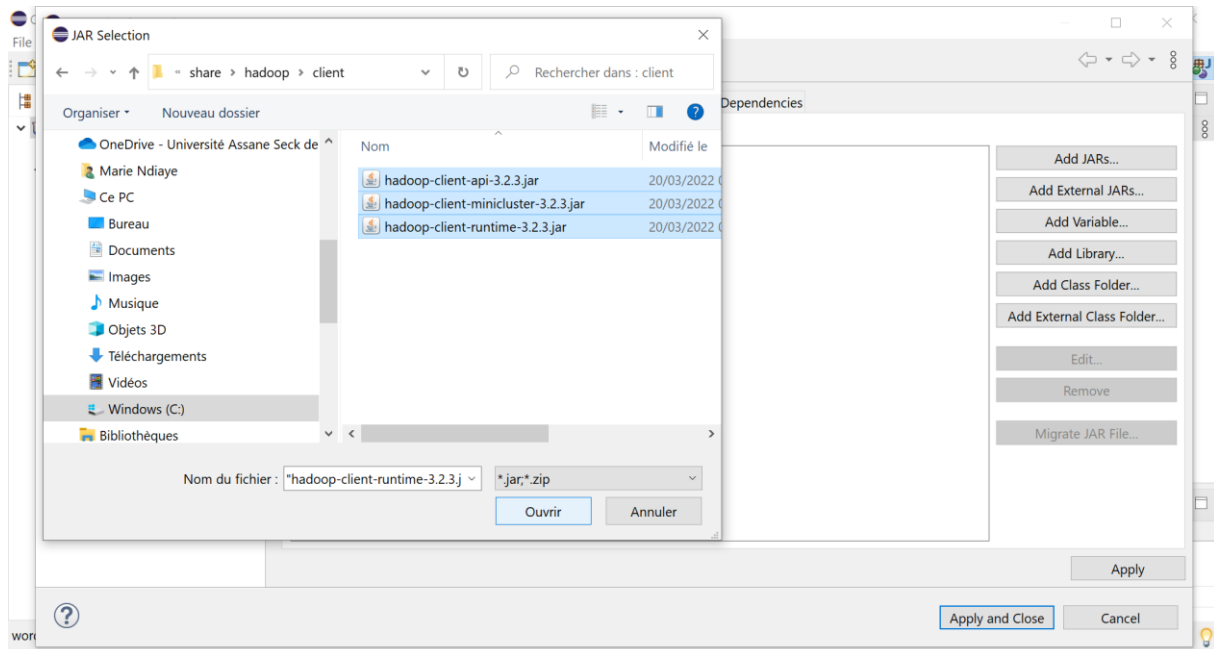


C:\hadoop\share\hadoop

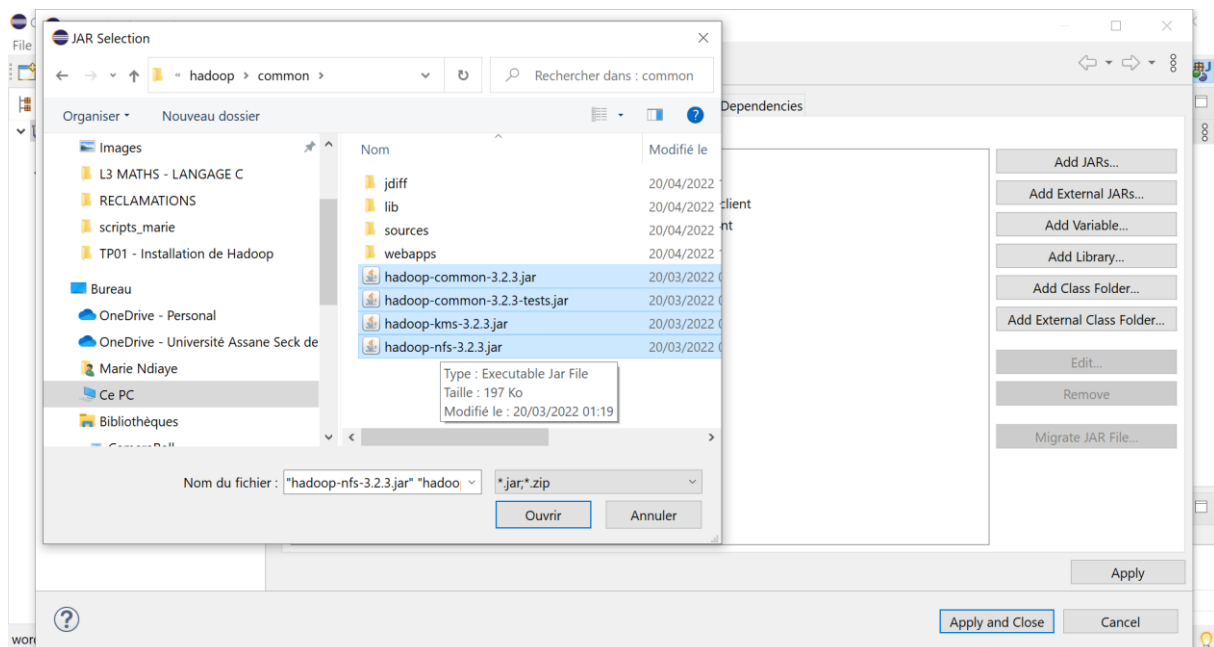




Client

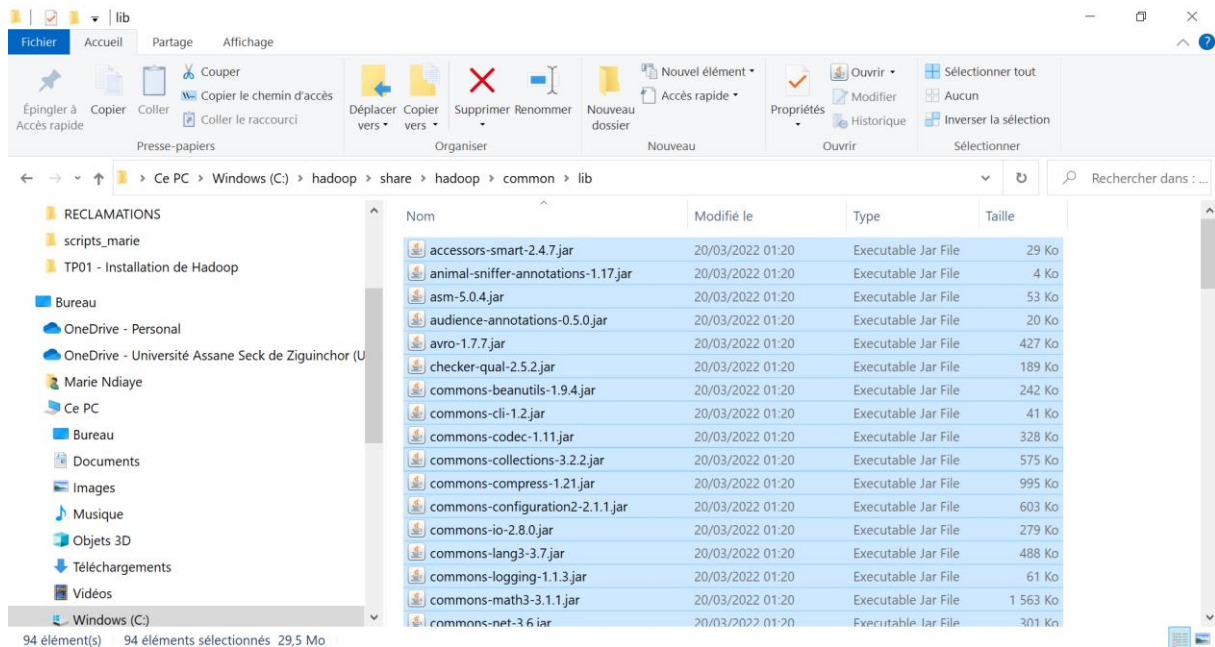


Common

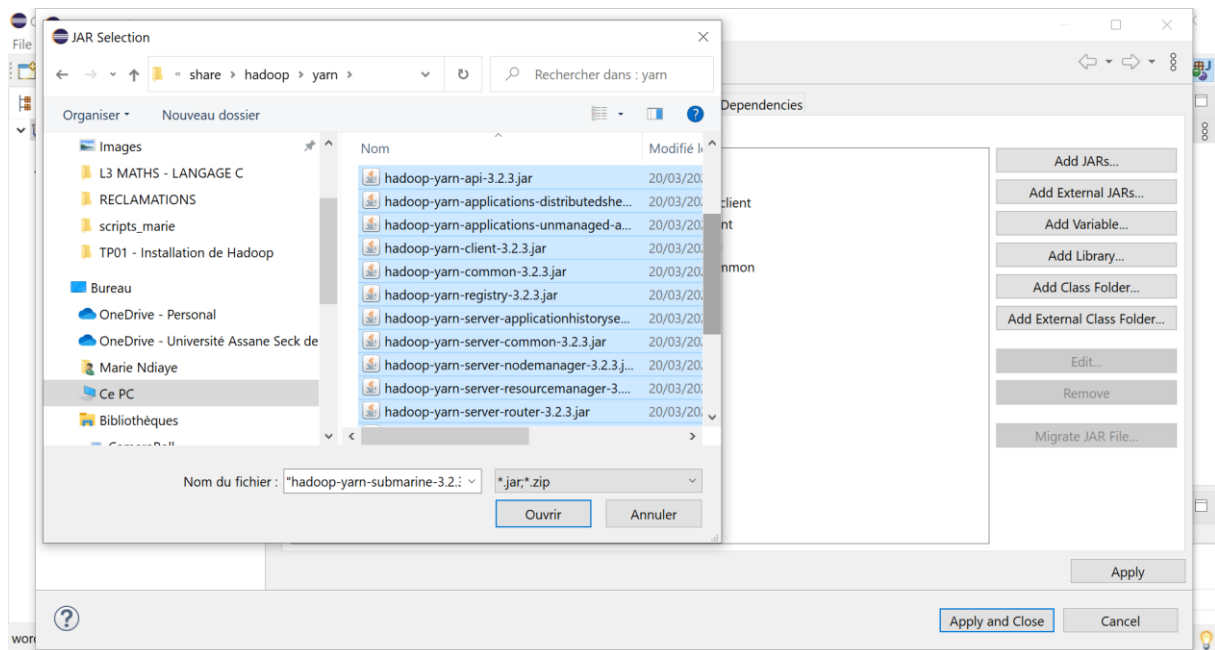




Common/lib

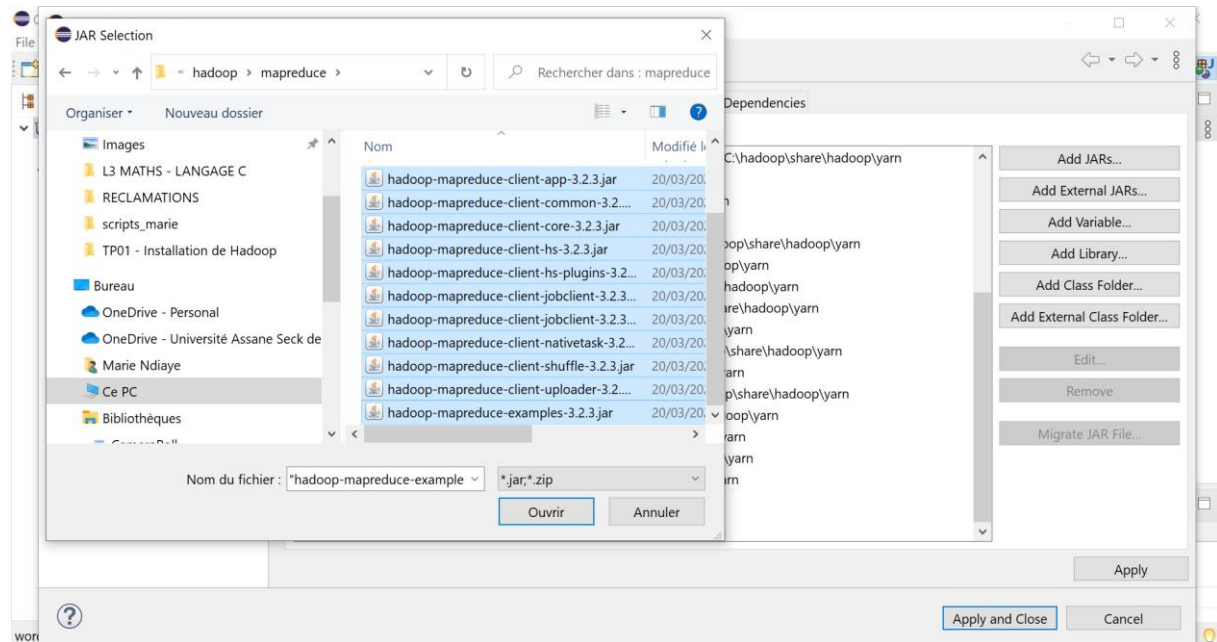


Yarn

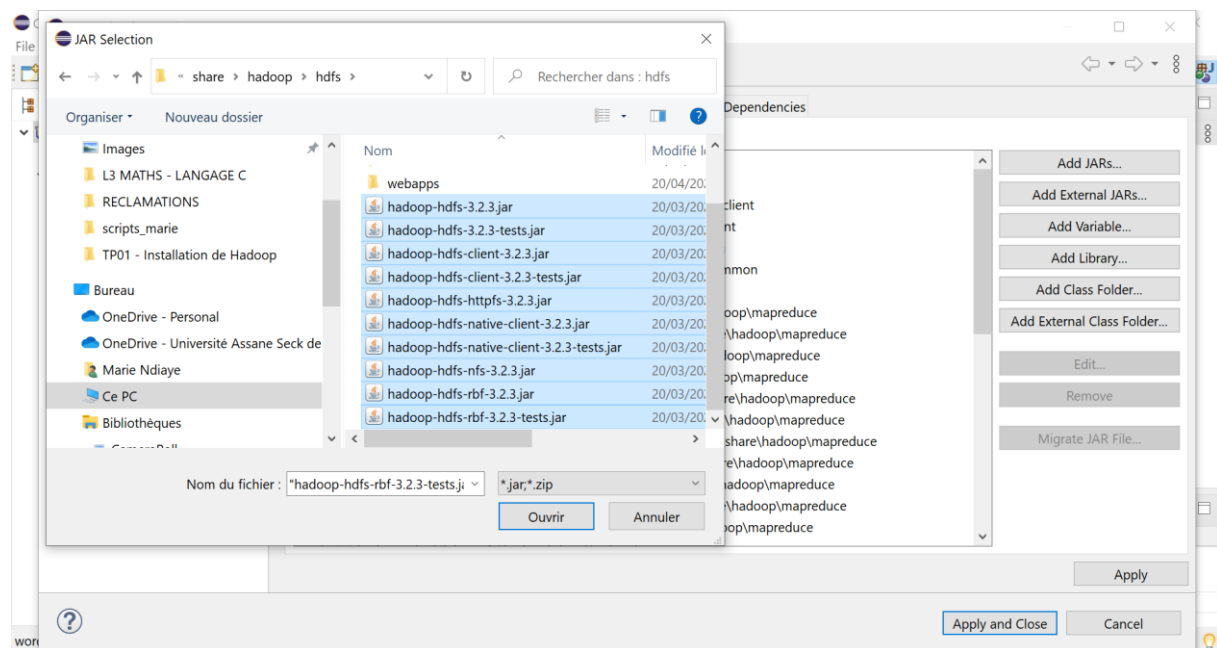




Mapreduce



Hdfs





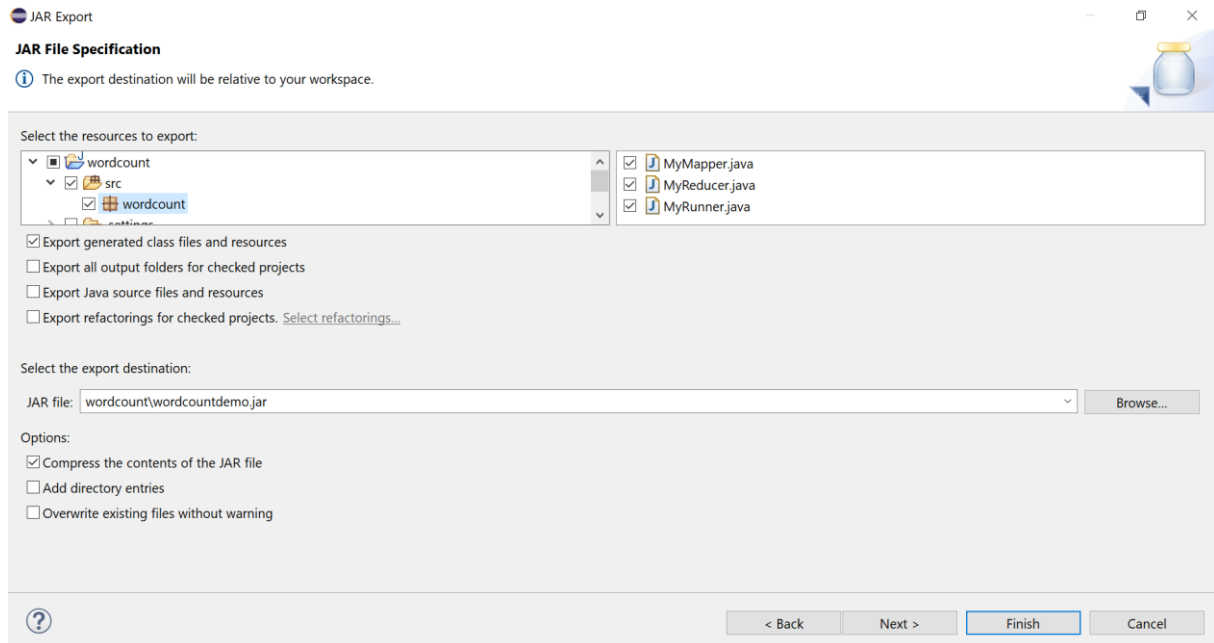
Créer le jar du code

The screenshot shows the Eclipse IDE interface. The top part displays the 'MyRunner.java' file with the following code:

```
import java.io.IOException;
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.Reducer;
import org.apache.hadoop.mapreduce.lib.input.TextInputFormat;
import org.apache.hadoop.mapreduce.lib.output.TextOutputFormat;

public class MyRunner {
    public static void main(String[] args) throws IOException {
        Configuration conf = new Configuration();
        Job job = new Job(conf, "WordCount");
        job.setJarByClass(MyRunner.class);
        job.setMapperClass(MyMapper.class);
        job.setReducerClass(MyReducer.class);
        job.setInputFormatClass(TextInputFormat.class);
        job.setOutputFormatClass(TextOutputFormat.class);
        job.setMapOutputKeyClass(Text.class);
        job.setMapOutputValueClass(IntWritable.class);
        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(IntWritable.class);
        job.run();
    }
}
```

The bottom part shows the 'Export' dialog box with the 'JAR file' option selected under the 'Java' category. The 'Next >' button is highlighted.



Soumettre le job

Dans l'invite de commande, taper la commande suivante :

```
hadoop jar C:\hadoop\jar\wordcountdemo.jar wordcount.MyRunner  
/test/data.txt /r_output
```

- **wordcountdemo.jar** : fichier .jar (programme)
- **wordcount.MyRunner** : Classe contenant la méthode main
- **/test/data.txt** : données sur lesquels le programme sera exécuté
- **/r_output** : Répertoire où la sortie du programme sera stockée



```
Administrateur : Invite de commandes - hadoop jar C:\hadoop\jar\wordcountdemo.jar wordcount.MyRunner /test/data.txt /r_output
C:\WINDOWS\system32>hadoop jar C:\hadoop\jar\wordcountdemo.jar wordcount.MyRunner /test/data.txt /r_output
2022-04-24 19:40:48,295 INFO client.DefaultNoHARMAFailoverProxyProvider: Connecting to ResourceManager at /0.0.0.0:8032
2022-04-24 19:40:48,922 INFO client.DefaultNoHARMAFailoverProxyProvider: Connecting to ResourceManager at /0.0.0.0:8032
2022-04-24 19:40:51,018 WARN mapreduce.JobResourceUploader: Hadoop command-line option parsing not performed. Implement the Tool interface and execute your application with ToolRunner to remedy this.
2022-04-24 19:40:51,120 INFO mapreduce.JobResourceUploader: Disabling Erasure Coding for path: /tmp/hadoop-yarn/staging/mariendiaye/.staging/job_1650825085064_0002
2022-04-24 19:40:51,837 INFO mapred.FileInputFormat: Total input files to process : 1
2022-04-24 19:40:52,162 INFO mapreduce.JobSubmitter: number of splits:2
2022-04-24 19:40:53,129 INFO mapreduce.JobSubmitter: Submitting tokens for job: job_1650825085064_0002
2022-04-24 19:40:53,132 INFO mapreduce.JobSubmitter: Executing with tokens: []
2022-04-24 19:40:53,798 INFO conf.Configuration: resource-types.xml not found
```

```
Administrateur : Invite de commandes - hadoop jar C:\hadoop\jar\wordcountdemo.jar wordcount.MyRunner /test/data.txt /r_output
2022-04-24 19:40:53,798 INFO conf.Configuration: resource-types.xml not found
2022-04-24 19:40:53,812 INFO resource.ResourceUtils: Unable to find 'resource-types.xml'.
2022-04-24 19:40:54,037 INFO impl.YarnClientImpl: Submitted application application_1650825085064_0002
2022-04-24 19:40:54,184 INFO mapreduce.Job: The url to track the job: http://LAPTOP-JUPTA8SR:8088/proxy/application_1650825085064_0002/
2022-04-24 19:40:54,245 INFO mapreduce.Job: Running job: job_1650825085064_0002
```

Autres commandes

- Lister les jobs en cours

```
mapred job -list
```




```
Administrateur : Invite de commandes

C:\WINDOWS\system32>mapred job -list
2022-04-24 19:40:53,544 INFO client.DefaultNoHARMFailoverProxyProvider: Connecting
to ResourceManager at /0.0.0.0:8032
2022-04-24 19:40:56,594 INFO conf.Configuration: resource-types.xml not found
2022-04-24 19:40:56,604 INFO resource.ResourceUtils: Unable to find 'resource-type
s.xml'.
Total jobs:1

JobId          JobName          State          StartTime
-----
User           Queue            Priority        UsedContainers  RsvdContainers  U
sedMem  RsvdMem    NeededMem      AM info
job_1650825085064_0002 WordCount        PREP          1650829253953  m
ariendiaye      default          DEFAULT        0              0
0M            0M              0M            http://LAPTOP-JUPTA8SR:8088/proxy/applicat
ion_1650825085064_0002/

C:\WINDOWS\system32>
```

- Voir un job en detail

http://LAPTOP-JUPTA8SR:8088/proxy/application_1650825085064_0002/

The screenshot shows the Hadoop web interface for application_1650825085064_0002. The interface is titled 'Application application_1650825085064_0002'. On the left, there is a sidebar with a 'Cluster' section containing links for 'About', 'Nodes', 'Node Labels', and 'Applications'. The 'Applications' link is selected, showing a list of application states: NEW, NEW_SAVING, SUBMITTED, ACCEPTED, RUNNING, FINISHED, FAILED, and KILLED. Below this is a 'Scheduler' section. The main content area displays application details for 'WordCount' (Application Type: MAPREDUCE). The details include: User: mariendiaye, Name: WordCount, Application Type: MAPREDUCE, Application Tags: (empty), Application Priority: 0 (Higher Integer value indicates higher priority), YarnApplicationState: ACCEPTED: waiting for AM container to be allocated, launched and register with RM., Queue: default, FinalStatus Reported by AM: Application has not completed yet., Started: dim. avr. 24 19:40:53 +0000 2022, Launched: N/A, Finished: N/A, Elapsed: 1mins, 24sec, Tracking URL: ApplicationMaster, Log Aggregation Status: DISABLED, and Application Timeout: Unlimited. There is a 'Kill Application' button at the top of the details section.

- Arrêter un job Hadoop

```
hadoop job -kill $jobId
```



```
Administrateur : Invite de commandes

C:\WINDOWS\system32>mapred job -kill job_1650825085064_0002
2022-04-24 19:46:16,573 INFO client.DefaultNoHARMAFailoverProxyProvider: Connecting to ResourceManager at /0.0.0.0:8032
2022-04-24 19:46:19,636 INFO impl.YarnClientImpl: Killed application application_1650825085064_0002
Killed job job_1650825085064_0002

C:\WINDOWS\system32>
```

Références

- <https://mbaron.developpez.com/tutoriels/bigdata/hadoop/introduction-hdfs-map-reduce/#LIII-A>
- Commandes site officiel : <https://hadoop.apache.org/docs/stable/hadoop-project-dist/hadoop-hdfs/HDFSCommands.html>
- <https://intellipaat.com/blog/tutorial/hadoop-tutorial/hdfs-operations/#2>
- Commandes dfs :
 - o <https://cdmana.com/2021/09/20210918200412792d.html>
 - o <https://hadoop.apache.org/docs/stable/hadoop-project-dist/hadoop-common/FileSystemShell.html>
- https://www.bogotobogo.com/Hadoop/BigData_hadoop_Running_MapReduce_Job.php
- Top 10 Hadoop HDFS Commands with Examples and Usage : <https://data-flair.training/blogs/top-hadoop-hdfs-commands-tutorial/>
- Hdfs user guide: <https://hadoop.apache.org/docs/r3.3.2/hadoop-project-dist/hadoop-hdfs/HdfsUserGuide.html>



Annexe 1 : Commandes dfs

Lister les fichiers

Syntaxe

```
$hadoop fs -ls [-c] [-d] [-h] [-R] [-t] [-S] [-r] [-u] /args  
ou  
$hdfs dfs -ls [-c] [-d] [-h] [-R] [-t] [-S] [-r] [-u] /args
```

Options

HDFS ls Options	Description
-c	Display the paths of files and directories only.
-d	Directories are listed as plain files.
-h	Formats the sizes of files in a human-readable fashion rather than several bytes.
-q	Print? instead of non-printable characters.
-R	Recursively list the contents of directories.
-t	Sort files by modification time (most recent first).
-S	Sort files by size.
-r	Reverse the order of the sort.
-u	Use the time of last access instead of modification for display and sorting.
-e	Display the erasure coding policy of files and directories.



UFR Sciences et Technologies
Département Informatique
MASTER 2 INFORMATIQUE
GENIE LOGICIEL
Enseignant : Pr Marie NDIAYE

BIG DATA
2024 - 2025
